

Decision Tree và Random Forest

Nguyen Minh Chu

Machine Learning & IoT Lab (ML IoT)
Trường Đại học Bách khoa, ĐHQG-HCM (HCMUT)

Nội dung bài giảng: Decision Tree, Ensemble Learning và Random Forest

Machine Learning & IoT Lab (ML IoT)
Ho Chi Minh City University of Technology (HCMUT)

1. Tổng quan về Cây quyết định (Decision Tree)
2. Tiêu chí toán học chia nhánh & Thuật toán mọc cây
3. Hiện tượng Overfitting và Kỹ thuật Pruning
4. Triết lý Học kết hợp (Ensemble Learning)
5. Rừng ngẫu nhiên (Random Forest) và Cơ chế Vận hành
6. Quy trình thực hành Pipeline và Tối ưu hóa Hyperparameter

Trọng tâm

Cân bằng giữa trực giác mô hình, công thức toán học và triển khai thực hành bằng Python.

Sau bài học này, sinh viên có thể

- ▶ Giải thích cấu trúc luận lý và cơ chế chia nhánh của Decision Tree.
- ▶ Tính toán thành thạo chỉ số Gini Impurity và Information Gain.
- ▶ Phân biệt bản chất toán học của hai chiến lược Bagging và Boosting.
- ▶ Hiểu sâu các cơ chế ngẫu nhiên hóa để xây dựng Random Forest robust.
- ▶ Triển khai pipeline phân loại và tinh chỉnh mô hình hoàn chỉnh.

Định nghĩa

Decision Tree là mô hình học máy có giám sát dạng phi tuyến, phân tách dữ liệu thành các tập con dựa trên chuỗi điều kiện logic.

Thành phần cấu trúc

- ▶ Root Node: chứa toàn bộ dữ liệu ban đầu.
- ▶ Decision Node: kiểm tra thuộc tính và ngưỡng.
- ▶ Leaf Node: chứa nhãn lớp hoặc giá trị dự đoán.
- ▶ Branch: kết quả điều kiện Đúng/Sai.

Ý nghĩa trực giác

- ▶ Gini Impurity đo xác suất một mẫu bị gán nhãn sai nếu nhãn được chọn ngẫu nhiên theo phân phối tại node.
- ▶ Độ tạp chất càng thấp, node càng thuần khiết và phép chia càng có giá trị.
- ▶ Khi node chỉ chứa một lớp duy nhất, Gini đạt giá trị tối thiểu.

Chiến lược chọn nhánh

CART ưu tiên phép tách làm giảm Gini Impurity mạnh nhất sau phân hoạch.

Gini tại một node

$$\text{Gini}(D) = 1 - \sum_{i=1}^C p_i^2$$

- ▶ C: tổng số lớp tại node đang xét.
- ▶ p_i : tỷ lệ mẫu thuộc lớp thứ i trong tập dữ liệu D.
- ▶ Giá trị càng gần 0 thì node càng thuần khiết.

Gini sau khi phân hoạch bởi thuộc tính A

$$\text{Gini}_A(D) = \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} \text{Gini}(D_v)$$

- ▶ D_v : tập con nhận giá trị hoặc nhánh v sau khi tách.
- ▶ Trọng số phản ánh tỷ lệ mẫu rơi vào từng nhánh con.
- ▶ Chọn phép tách có Weighted Gini Index nhỏ nhất.

Entropy

- ▶ Entropy đo độ hỗn loạn thông tin hoặc mức độ không chắc chắn của node.
- ▶ Node thuần khiết có Entropy thấp nhất.
- ▶ Node có phân phối lớp càng đều thì Entropy càng cao.

Information Gain

Information Gain đo lượng Entropy giảm được sau khi chia dữ liệu theo một thuộc tính.

Entropy tại một node

$$\text{Entropy}(D) = - \sum_{i=1}^C p_i \log_2(p_i)$$

- ▶ p_i : tỷ lệ mẫu thuộc lớp thứ i .
- ▶ Entropy bằng 0 khi node thuần khiết hoàn toàn.
- ▶ Entropy tăng khi phân phối nhãn trong node cân bằng hơn.

Mức giảm Entropy sau khi tách bởi thuộc tính A

$$IG(D, A) = Entropy(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} Entropy(D_v)$$

- ▶ Thuật toán quét các thuộc tính và ngưỡng cắt khả dĩ.
- ▶ Thuộc tính có Information Gain lớn nhất được ưu tiên làm điểm chia.

Thuật toán	Tiêu chí	Đặc điểm chính
ID3	Information Gain	Dùng cho biến categorical, chia nhiều nhánh; thiên vị thuộc tính có nhiều giá trị.
C4.5	Gain Ratio	Chuẩn hóa Information Gain, xử lý biến liên tục và missing values.
CART	Gini / Variance Reduction	Cây nhị phân; dùng Gini cho phân loại và giảm phương sai cho hồi quy.

Vai trò

Cây có xu hướng mọc sâu đến khi các lá thuần khiết; cần siêu tham số để tránh Overfitting.

- ▶ `max_depth`: giới hạn số tầng tối đa từ Root Node đến Leaf Node sâu nhất.
- ▶ `min_samples_split`: số mẫu tối thiểu để node nội bộ được phép tiếp tục tách.
- ▶ `min_samples_leaf`: số mẫu tối thiểu bắt buộc tồn tại ở Leaf Node sau phép chia.

Ghi nhớ

Ràng buộc quá lỏng gây Overfitting; ràng buộc quá chặt gây Underfitting.

Pre-pruning

- ▶ Dừng mọc cây ngay trong lúc huấn luyện.
- ▶ Cấu hình chặt max_depth, min_samples_leaf.
- ▶ Giảm Overfitting sớm, chi phí thấp.

Post-pruning

- ▶ Cho cây phát triển tự do trước.
- ▶ Duyệt ngược từ dưới lên để cắt nhánh yếu.
- ▶ Tối ưu theo Cost Complexity Pruning.

Hàm mục tiêu sau cắt tỉa

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

- ▶ $R(T)$: sai số của cây trên dữ liệu đánh giá.
- ▶ $|T|$: số lượng Leaf Node trong cây.
- ▶ α : hệ số phạt độ phức tạp kiến trúc.

Ưu điểm mạnh mẽ

- ▶ White-box: quy tắc logic rõ ràng và dễ giải thích.
- ▶ Không cần Feature Scaling như MinMax hoặc Z-score.
- ▶ Linh hoạt với cả dữ liệu số và dữ liệu định danh.

Nhược điểm chí mạng

- ▶ Dễ Overfitting nếu thiếu Pruning.
- ▶ High Variance: nhạy với biến động nhỏ trong dữ liệu train.
- ▶ Cấu trúc cây có thể thay đổi mạnh do nhiễu nhỏ.

Triết lý Wisdom of the crowd

- ▶ Không đặt cược vào duy nhất một mô hình mạnh nhưng bất ổn.
- ▶ Kết hợp nhiều weak learners độc lập để tạo quyết định cuối cùng.
- ▶ Sai số ngẫu nhiên và thiên lệch cục bộ có xu hướng triệt tiêu khi số mô hình đủ lớn.

Kết quả

Ensemble Learning giúp giảm Variance hoặc Bias tổng thể của hệ thống dự đoán.

Bagging

- ▶ Bootstrap Aggregating.
- ▶ Huấn luyện song song và độc lập.
- ▶ Mục tiêu: giảm Variance, chống Overfitting.
- ▶ Ví dụ tiêu biểu: Random Forest.

Boosting

- ▶ Huấn luyện tuần tự nối tiếp.
- ▶ Mô hình sau sửa sai mô hình trước.
- ▶ Mục tiêu: giảm Bias.
- ▶ Ví dụ: AdaBoost, XGBoost, LightGBM.

Bản chất kiến trúc

Random Forest là Ensemble dựa trên Bagging, gồm nhiều cây quyết định CART hoạt động độc lập.

1. Với tập huấn luyện kích thước N , lấy mẫu ngẫu nhiên N lần có hoàn lại để tạo một Bootstrap dataset.
2. Một số mẫu xuất hiện nhiều lần; một số mẫu hoàn toàn vắng mặt.
3. Lặp lại B lần để tạo B tập Bootstrap cho B cây trong rừng.

Vấn đề của Bagging thuần túy

Dominant features có thể khiến nhiều cây chọn cùng thuộc tính ở node gốc, làm tương quan giữa cây tăng cao.

- ▶ Tại mỗi node, thuật toán không tìm kiếm trên toàn bộ tập thuộc tính ban đầu.
- ▶ Chọn ngẫu nhiên một tập con thuộc tính để xét điểm chia tối ưu.
- ▶ Cơ chế này làm giảm tương quan giữa các cây trong rừng.
- ▶ Độ đa dạng cao giúp Ensemble giảm Variance tốt hơn.

Khuyến nghị cho bài toán Classification

$$m = \sqrt{M}$$

- ▶ M: tổng số thuộc tính ban đầu.
- ▶ m: số thuộc tính được chọn ngẫu nhiên tại mỗi node.
- ▶ Việc chỉ xét m thuộc tính giúp bẻ gãy tương quan giữa các cây.

Classification

- ▶ Mỗi cây bỏ phiếu cho một nhãn lớp.
- ▶ Majority Voting chọn nhãn có nhiều phiếu nhất.
- ▶ Dự đoán cuối giảm nhiễu của từng cây đơn lẻ.

Regression

- ▶ Mỗi cây dự đoán một giá trị liên tục.
- ▶ Average lấy trung bình cộng toàn bộ B cây.
- ▶ Giảm dao động dự đoán trên dữ liệu nhiễu.

Khái niệm OOB

- ▶ Với Bootstrap Sampling, một phần dữ liệu gốc không xuất hiện trong tập huấn luyện của từng cây.
- ▶ Các mẫu vắng mặt này gọi là Out-of-Bag samples.
- ▶ OOB samples có thể dùng như tập đánh giá nội bộ cho từng cây.

Ứng dụng đánh giá

Dự đoán mỗi mẫu bằng các cây không dùng mẫu đó để huấn luyện, từ đó ước lượng hiệu năng mà không cần Validation riêng.

Xác suất một mẫu không được chọn

$$\lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \approx 0.368$$

- ▶ Xấp xỉ 36.8% dữ liệu gốc không xuất hiện trong bootstrap của một cây.
- ▶ Đây là cơ sở toán học cho OOB Error trong Random Forest.

Khái niệm

- ▶ Đánh giá mức đóng góp của từng thuộc tính vào sức mạnh dự đoán chung của Random Forest.
- ▶ Thuộc tính gần gốc và làm sạch dữ liệu mạnh có điểm Importance cao.
- ▶ Cách đo nhanh nhưng có thể thiên vị thuộc tính nhiều mức phân biệt.

Mean Decrease Gini

$$\text{Importance}(A) = \sum_{t \in \text{Trees}} \sum_{n \in \text{Split}(A)} \Delta \text{Gini}(n, A)$$

- ▶ Tổng hợp mức giảm Gini do thuộc tính A tạo ra trên toàn bộ cây.
- ▶ Điểm càng cao thì thuộc tính càng đóng góp mạnh vào phân tách dữ liệu.

Cơ chế đo lường hoán vị

1. Xáo trộn ngẫu nhiên riêng thuộc tính A trên OOB, giữ nguyên các thuộc tính khác.
2. Đưa dữ liệu đã hoán vị qua mô hình và đánh giá lại Accuracy.
3. Accuracy giảm mạnh chứng tỏ thuộc tính A rất quan trọng.

Ý nghĩa

Đo mức phụ thuộc thực nghiệm của mô hình vào tín hiệu thật của từng thuộc tính.

Thực hành

Hiện thực cây quyết định from scratch, dùng tập dữ liệu Iris.